

챕터 4·5 실습 환경 사용 가이드

0. 시작 전 — 빌드와 환경 점검 (최초 1회)
 - 점검 결과 읽는 법
1. 앞으로의 실습 환경 — 큰 그림
2. 매일 실습 시작하기
 - Step 1 — Docker Desktop 앱 실행
 - Step 2 — 컨테이너 상태 확인
 - Step 3 — JupyterLab 접속
3. 실습 끝날 때 — 그냥 두기!
 - ✅ 권장 방법: 끄지 말고 그냥 켜두기
 - ⏸ 잠깐 멈추고 싶을 때: Stop (중지)
 - ⚠ 주의: `docker compose down` 은 되도록 사용하지 마세요
4. 컨테이너를 Down 해야 하는 경우
 - 어떤 경우에 Down이 필요한가요?
 - Down => 재시작 절차
5. 패키지 목록이 업데이트되었을 때 (재빌드)
6. 컨테이너가 사라졌을 때 (재생성)
7. 한눈에 보는 요약표
8. 실습 중 유용한 명령어
 - 환경 점검 (문제가 생겼을 때 가장 먼저)
 - 컨테이너 내부 터미널 접속
 - GPU 상태 확인 (컨테이너 내부에서)
 - 컨테이너 내부에서 나오기
 - 컨테이너 로그 확인
9. 기억해야 할 핵심 개념 4가지
 - ① Docker Desktop 앱이 **리모컨**입니다
 - ② 파일은 어디에 저장되나요?
 - ③ PowerShell vs 컨테이너 내부 — 어디서 뭘 하나요?
 - ④ 실습 노트북 첫 줄 규칙 (챕터 5)

챕터 4·5 실습 환경 사용 가이드

이 문서는 뭔가요?

환경설정(드라이버 => Docker Desktop => WSL => 빌드)이 모두 끝난 뒤, 앞으로 매일 실습할 때 어떻게 사용하면 되는지 정리한 문서입니다.

0. 시작 전 — 빌드와 환경 점검 (최초 1회)

환경설정 마지막 두 단계입니다. 이미 마치셨다면 1장으로 넘어가세요.

PowerShell을 열고 실습 폴더에서 아래를 실행합니다.

```
cd ~
cd ssafy_ai_2

# 1. 이미지 빌드 + 컨테이너 실행 (처음 한 번만 오래 걸립니다)
docker compose up -d --build

# 2. 환경 점검
docker compose run --rm lab verify_env.sh
```

💡 `-d`는 백그라운드 실행 옵션입니다. 이걸 빼면 PowerShell 창이 로그로 계속 점유되고, 실수로 `Ctrl + C`를 누르면 컨테이너가 멈춥니다.

점검 결과 읽는 법

`verify_env.sh`는 6개 항목을 순서대로 검사합니다.

모든 검증 통과. 실습을 시작해도 좋습니다.

이 문구가 마지막에 나오면 완료입니다. `[FAIL]`이 하나라도 있으면 아래 표를 확인하세요.

실패 항목	흔한 원인	조치
2. PyTorch / GPU 에서 CUDA 미인식	GPU 할당 실패 또는 드라이버 문제	PowerShell에서 <code>nvidia-smi</code> 가 되는지 확인 => 안 되면 드라이버 재설치
4. 4bit 양자화 실패	이미지 빌드가 중간에 꼬임	<code>docker compose build --no-cache</code> 로 재빌드
5. 도구 계층 실패	패키지 설치 누락	위와 동일하게 재빌드
그 외	—	로그 전체를 캡처해서 강사에게 문의

빌드 자체가 실패했다면, 대부분 **Docker Desktop의 WSL integration 체크가 빠진 경우**입니다. 설정에서 먼저 확인해 주세요.

1. 앞으로의 실습 환경 — 큰 그림

빌드가 끝났으므로, 이제부터는 **Docker Desktop 앱**을 통해 실습 환경을 관리합니다.

Docker Desktop 앱 실행 => 컨테이너 켜짐 확인 => 브라우저에서 JupyterLab 접속 => 실습!

복잡한 명령어를 외울 필요 없이, **Docker Desktop 앱 화면**에서 컨테이너 상태를 눈으로 확인하면 됩니다.

2. 매일 실습 시작하기

Step 1 — Docker Desktop 앱 실행

바탕화면의 **Docker Desktop** 아이콘을 더블클릭합니다.
(또는 `Windows + S` => "Docker" 검색 => 실행)

Step 2 — 컨테이너 상태 확인

Docker Desktop 앱 왼쪽 메뉴에서 **Containers**를 클릭합니다.

상태	의미	해야 할 일
 Running	컨테이너가 실행 중	바로 Step 3으로 이동
 Stopped	컨테이너가 중지됨	 재생 버튼 클릭하여 시작
아무것도 안 보임	컨테이너가 삭제된 상태 (<code>down</code> 된 경우)	아래 컨테이너 재생성 참고

Step 3 — JupyterLab 접속

컨테이너가 **Running** 상태라면, 브라우저를 열고 아래 주소를 입력합니다.

항목	값
URL	<code>http://localhost:8888</code>
Token	<code>lab</code>

왼쪽 파일 탐색기에 실습/과제 파일이 보이면 **실습 준비 완료**입니다!

3. 실습 끝날 때 — 그냥 두기!

✅ 권장 방법: 끄지 말고 그냥 켜두기

앞으로는 특수한 경우가 아니라면 컨테이너를 종료하지 않고 켜둔 상태로 사용하시면 됩니다.

컨테이너를 켜둔 채로 컴퓨터를 종료하거나 재시작해도,
다음에 Docker Desktop이 실행되면 **컨테이너가 자동으로 다시 켜집니다**.

즉, 한 번 켜두면 매일 이렇게만 하면 됩니다.

Docker Desktop 실행 => 브라우저에서 localhost:8888 접속 => 끝!

▣ 잠깐 멈추고 싶을 때: Stop (중지)

리소스를 아끼고 싶거나 PC가 느려질 때는 **Stop**으로 중지할 수 있습니다.

Docker Desktop 앱 => Containers => 해당 컨테이너의 **Stop** 버튼 클릭

다시 시작할 때는 **Start** 버튼만 누르면 됩니다.

⊖ 주의: `docker compose down`은 되도록 사용하지 마세요

`down` 명령을 실행하면 컨테이너가 **삭제**되어 Docker Desktop 앱의 Containers 목록에서 **사라집니다**.

다시 만들려면 PowerShell에서 명령어를 입력해야 하므로 불편합니다.

동작	명령/조작	컨테이너 상태	앱에서 보이나요?
켜두기 (권장)	아무것도 안 함	Running	✅ 보임
중지 (Stop)	앱에서 Stop 클릭	Stopped	✅ 보임
삭제 (Down)	<code>docker compose down</code>	삭제됨	❌ 안 보임

안심하세요. `down`을 하더라도 작업 파일은 사라지지 않습니다.

Windows 폴더와 연결(볼륨)되어 있기 때문입니다.

4. 컨테이너를 Down 해야 하는 경우

대부분은 켜둔 채로 사용하면 되지만, 아래 상황에서는 컨테이너를 완전히 끄고 재시작해야 합니다.

어떤 경우에 Down이 필요한가요?

- `.env` 파일 등 환경변수 파일을 추가하거나 수정했을 때
- `compose.yaml` 파일을 수정했을 때
- 강사가 패키지 목록(`requirements.lock.txt`)이나 `Dockerfile`을 업데이트했을 때

💡 왜? 환경변수와 Docker 설정 파일은 컨테이너가 처음 만들어질 때 읽어옵니다.

이미 실행 중인 컨테이너에는 변경 사항이 반영되지 않기 때문에,
완전히 제거하고 새로 만들어야 합니다.

Down => 재시작 절차

PowerShell을 열고 `ssafy_ai_2` 폴더에서 아래 명령어를 순서대로 실행합니다.

```
# 1. 실습 폴더로 이동
cd ~
cd ssafy_ai_2

# 2. 컨테이너 완전 제거
docker compose down

# 3. 컨테이너 재생성 & 백그라운드 실행
docker compose up -d
```

실행 후 Docker Desktop 앱 => Containers에서 컨테이너가 다시  **Running** 상태로 보이면 성공입니다.

5. 패키지 목록이 업데이트되었을 때 (재빌드)

강사가 `requirements.lock.txt` 또는 `Dockerfile` 을 업데이트했다고 안내하면, 이미지를 다시 만들어야 합니다. 위의 `up -d` 만으로는 반영되지 않습니다.

```
cd ~
cd ssafy_ai_2

# 1. 업데이트된 설정 파일 받기
git pull

# 2. 이미지 재빌드 + 재실행
docker compose up -d --build

# 3. 환경 재점검
docker compose run --rm lab verify_env.sh
```

몇 분 정도 걸릴 수 있습니다. 3번까지 통과하면 정상입니다.

 컨테이너 내부에서 `pip install` 로 직접 설치한 패키지는 재빌드하면 **사라집니다**. 이건 버그가 아니라 의도된 동작입니다. 모두가 같은 환경을 쓰기 위한 구조이므로, 계속 필요한 패키지가 있다면 강사님에게 요청해 주세요.

6. 컨테이너가 사라졌을 때 (재생성)

`docker compose down` 을 실행하거나 실수로 컨테이너를 삭제한 경우, Docker Desktop 앱의 Containers 목록에 아무것도 보이지 않습니다.

이때는 **PowerShell**에서 다시 만들어주면 됩니다.

```
cd ~
cd ssafy_ai_2
docker compose up -d
```

이미지는 그대로 남아있으므로 **빌드 없이 금방 끝납니다**. 이후 Docker Desktop 앱에서 컨테이너가 다시 나타나고, `localhost:8888` 로 접속할 수 있습니다.

7. 한눈에 보는 요약표

상황	어떻게 하나요?
매일 실습 시작	Docker Desktop 앱 실행 => 브라우저에서 <code>localhost:8888</code> 접속
실습 끝	그냥 켜두면 됨 (종료 불필요)
PC 느려질 때	Docker Desktop 앱에서  Stop 클릭
다시 시작	Docker Desktop 앱에서  Start 클릭
환경변수 파일 변경 시	<code>docker compose down</code> => <code>docker compose up -d</code>
패키지 목록 업데이트 안내를 받았을 때	<code>git pull</code> => <code>docker compose up -d --build</code>
컨테이너가 목록에서 사라졌을 때	<code>docker compose up -d</code>
환경이 이상할 때	<code>docker compose run --rm lab verify_env.sh</code>

8. 실습 중 유용한 명령어

아래는 특수한 상황에서만 필요한 명령어입니다. 평소에는 쓸 일이 거의 없습니다.

환경 점검 (문제가 생겼을 때 가장 먼저)

```
docker compose run --rm lab verify_env.sh
```

Python 버전, GPU 인식, 4bit 양자화, 주요 라이브러리를 한 번에 점검합니다.
강사에게 문의할 때 이 결과를 함께 보내주시면 원인 파악이 훨씬 빠릅니다.

컨테이너 내부 터미널 접속

```
docker compose exec lab bash
```

컨테이너 안으로 들어가면 프롬프트가 `root@...:/workspace#` 으로 바뀝니다.

GPU 상태 확인 (컨테이너 내부에서)

```
nvidia-smi
```

학습 중 VRAM 사용량을 확인할 때 유용합니다.

컨테이너 내부에서 나오기

```
exit
```

컨테이너 로그 확인

```
docker compose logs -f lab
```

JupyterLab이 접속되지 않을 때 원인을 확인할 수 있습니다. (Ctrl + C로 종료)

9. 기억해야 할 핵심 개념 4가지

① Docker Desktop 앱이 리모컨입니다

PowerShell 명령어를 외울 필요 없이, Docker Desktop 앱에서 **Start/Stop 버튼**으로 컨테이너를 관리할 수 있습니다. 컨테이너 상태(Running/Stopped)도 앱에서 바로 확인됩니다.

② 파일은 어디에 저장되나요?

JupyterLab에서 작업하는 파일은 Windows의 `C:\Users\SSAFY\ssafy_ai_2\` 폴더와 실시간 동기화됩니다.
컨테이너를 Stop하거나 Down해도 작업 파일은 삭제되지 않습니다.

단, `/workspace` 바깥에 저장한 파일은 동기화 대상이 아니므로 사라집니다. 항상 왼쪽 파일 탐색기에 보이는 위치에 저장하세요.

③ PowerShell vs 컨테이너 내부 — 어디서 뭘 하나요?

환경	하는 일	프롬프트 모양
PowerShell	Docker 명령어 (up, down, exec, run)	PS C:\Users\SSAFY\ssafy_ai_2>
컨테이너 내부	Linux 명령어, Python 실행, GPU 확인	root@xxxx:/workspace#

④ 실습 노트북 첫 줄 규칙 (챕터 5)

파인튜닝 실습에서는 `import unsloth` 를 반드시 최상단에 두어야 합니다.

```
import unsloth # 반드시 첫 줄. transformers / trl / peft 보다 먼저!  
  
from unsloth import FastLanguageModel
```

순서가 바뀌면 최적화가 적용되지 않아 학습이 느려지거나 메모리 부족 오류가 발생합니다.

🔴 매일 기억할 것은 딱 하나:

Docker Desktop 실행 => 브라우저에서 `localhost:8888` => 실습 시작!